Introduction

- SQL Constraints are used to specify rules for the data in the table, to ensure that the data in the table conforms to the requirements defined.
- Constraints are very important to maintain the data integrity.
- Constraints can be defined when the table is created (with the CREATE statement) or can be added after the table is created (with the ALTER statement).
- Every constraint is provided with a name (CONSTRAINT_NAME) in the database and name can be given when it is defined. In case we do not give a name to the constraint when we declare the constraint, Oracle associates the constraint with a name.
- Within the same scope, no two objects can have same name. Hence one user, can not use a name as constraint name if it is already used for some other object.

Constraint Types

There are six types of constraints available to provide rules to the tables.

- PRIMARY KEY
- UNIQUE
- FOREIGN KEY
- CHECK
- NOT NULL
- DEFAULT

Declaration Style

We can define constraints for a column in 2 different ways

1. **Column Level (OR) In-line style** : Constraint is defined as part of the definition of an individual column or attribute. Constraints can be defined at column level using both the CREATE and ALTER statements. They are usually specified when the constraint is specific to the column only.

Syntax:
CREATE TABLE sample (col1 DATATYPE(size) CONSTRAINT_NAME,col2 DATATYPE(size)
CONSTRAINT_NAME,…….);

2. **Table Level (OR) Out-of-lilne style** : Constraint is defined as part of the table definition. Constraints can be defined at table level using CREATE statement. They are usually specified when the constraint need to be applied on the combination of columns together.

Syntax:
CREATE TABLE sample
(col1 DATATYPE(size), col2
DATATYPE(size),col3DATATYPE(size),CONSTRAINT_NAME(col1),
CONSTRAINT_NAME(col2),…….);

Primary Key

- A PRIMARY KEY constraint uniquely identifies each record in the table.
- A PRIMARY KEY column cannot have duplicate values.
- A PRIMARY KEY column cannot have null values.
- A PRIMARY KEY can consist of one or more columns.
- If a single column does not qualify for a PRIMARY KEY, we can define a primary key by combining more than one columns.This is called COMPOSITE PRIMARY KEY.

Restrictions

- A table can have only one PRIMARY KEY.
- PRIMARY KEY cannot be defined on the columns having LOB,LONG,LONG RAW,BFILE etc... data types.
- A Composite PRIMARY KEY can be defined only as a table level key.
- A composite PRIMARY KEY cannot have more than 32 columns.

Primary key creation with Create table (Column Level) statement:

Syntax :
CREATE TABLE table1 (col1 DATATYPE(size) CONSTRAINT cons_name PRIMARY KEY, col2 DATATYPE(size), col3 DATATYPE(size),.....);

Ex: CREATE TABLE Location(loc_id NUMBER(4) CONSTRAINT loc_pk PRIMARY KEY, street_address VARCHAR2(20), city VARCHAR2(15), state VARCHAR2(15), pincode NUMBER(6));

The above example specifies that every location must have a unique location_id value to identify the location and that should not be left blank.

Primary key with Create statement (Table Level)
A PRIMARY KEY can be defined at the table level once the attributes of the table are declared .

Syntax :

CREATE TABLE table1 (col1 DATATYPE(size), col2 DATATYPE(size), col3 DATATYPE(size), ..., CONSTRAINT cons_name PRIMARY KEY(col1));

Ex: CREATE TABLE location (loc_id NUMBER(4), street_address VARCHAR2(20), city VARCHAR2(15), state VARCHAR2(15), pincode NUMBER(6) CONSTRAINT loc_pk PRIMARY KEY(loc_id));

The above example specifies that every location must have a unique value for location_id and that should not be left blank.

Composite Primary key

A PRIMARY KEY can be defined on multiple columns if required. This type of defining on multiple columns is called as composite PRIMARY KEY.

Syntax :

CREATE TABLE table1 (col1 DATATYPE(size), col2 DATATYPE(size), ..., CONSTRAINT cons_name PRIMARY KEY(col1,col2));
Example:

CREATE TABLE sales (sales_id NUMBER(6), cust_id NUMBER(4), prod_id NUMBER(5), qnty NUMBER(6), sold_date DATE, CONSTRAINT sales_pk PRIMARY KEY(sales_id,cust_id,prod_id));

The above example specifies that, for each record in this table , the combination of the following three fields sales_id , cust_id and prod_id will only give a unique value but not the value of only one of these fields can be unique .This is an example of a composite PRIMARY KEY .

Primary key with Alter statement
Using ALTER statement we can create a Primary Key for a table already created.

Syntax:

ALTER TABLE table_name ADD CONSRAINT cons_name PRIMARY KEY(colname);

Ex:

CREATE TABLE person (aadhar_id VARCHAR2(10), name VARCHAR2(30), dob DATE, phone_number NUMBER(12), address VARCHAR2(30));

ALTER TABLE person ADD CONSTRAINT aadhar_pk PRIMARY KEY (aadhar_id);

Removing Primary Key:

We can remove the primary key constraint from a table.

In order to remove a PRIMARY KEY constraint which is defined on a table , we need to use DROP CONSTRAINT with ALTER Table

Syntax:

ALTER TABLE table1 DROP CONSTRAINT cons_name;
Ex: ALTER TABLE location DROP CONSTRAINT loc_pk;

Foreign Key

- A FOREIGN KEY means establishing the relationship between parent and child tables. (parent – PRIMARY / UNIQUE defined table , child – FOREIGN KEY defined table)
- A FOREIGN KEY in one table points to a PRIMARY KEY in another table.
- The PRIMARY KEY and FOREIGN KEY can exist in the same table.
- A composite FOREIGN KEY should be defined at table level. A composite FOREIGN KEY must refer to the composite PRIMARY KEYcolumn.

Restrictions

- The child and parent tables must be in the same database.
- The column to be declared as FOREIGN KEY in child table must be of same data type as the data type of the PRIMARY KEY column of the parent table.

Foreign key constraint is called the Referential Integrity Constraint.

- It preserves the defined relationship between the tables when the records are inserted or deleted.
- It ensures that key values are consistent across the table

When referential integrity (Foreign Key) is enforced , it prevents from

1. Adding records to a child table if there is no associated record available in the parent table.
2. Changing values in the parent table is associated records available in the child table.

3. Deleting records from the parent table if there are matching records available in the child table.

Foreign key with Create statement -Column Level

A FOREIGN KEY constraint can be defined on a table with the help of a CREATE statement immediately after the column definition with the help of REFERENCES keyword to refer the parent table.

Syntax :

CREATE TABLE table_name (col_name1 DATATYPE(size) CONSTRAINT cons_name REFERENCES parent_table_name(col1), col5 DATATYPE(size), col6 DATATYPE(size) , ...);

Example :

CREATE TABLE person(name VARCHAR2(30),dob DATE,gender CHAR(1),address NUMBER(4) REFERENCES location(loc_id),gender CHAR(1));

The above example specifies that the address column values in the person table are linked/references with the column: **loc_id** values of the location table.

Foreign key with Create statement -Table Level

A FOREIGN KEY constraint can be defined as a table level definition once finishing all the column definitions.
Syntax :

CREATE TABLE table2 (col4 DATATYPE(size), col5 DATATYPE(size), col6 DATATYPE(size), ..., CONSTRAINT cons_name FOREIGN KEY(col4) REFERENCES table1(col1));

Ex: CREATE TABLE person(name VARCHAR2(30),dob DATE,gender CHAR(1),address NUMBER(4),gender CHAR(1),FOREIGN KEY(address) REFERENCES location(loc_id));

The above example specifies that in order to maintain the address for each person, the addresses available in the location table must be linked to the address field of the person table.

Composite Foreign key

We can define a FOREIGN KEY constraint multiple columns. This type of defining on multiple columns is called as composite FOREIGN KEY.

Syntax :

CREATE TABLE table2 (col4 DATATYPE(size), col5 DATATYPE(size), col6 DATATYPE(size), ..., CONSTRAINT cons_name FOREIGN KEY (col4,col5) REFERENCES table1(col1,col2));

Ex: CREATE TABLE production(suppl_id NUMBER(4),warehouse_id NUMBER(6), warehouse_name VARCHAR2(20),product_name VARCHAR2(20),qnty NUMBER(4),unit_price NUMBER(6,2),CONSTRAINT suppl_fk FOREIGN KEY(suppl_id) REFERENCES supplied(supp_id),CONSTRAINT warehouse_fk (warehouse_id,warehouse_name) REFERENCES (id,name));

Foreign key with Alter statement

We can use ALTER statement to add a Foreign Key to an already created table.
Syntax:

ALTER TABLE table2 ADD CONSRAINT cons_name FOREIGN KEY (col4) REFERENCES table1(col1);

Ex: CREATE TABLE person(name VARCHAR2(30),dob DATE,gender CHAR(1),address NUMBER(4) ,gender CHAR(1));

ALTER TABLE person ADD CONSTRAINT location_fk FOREIGN KEY(address)
REFERENCES location(loc_id);

Removing Foreign Key:

We can remove the foreign key of a table by using the ALTER table and Drop Constraint command.
Syntax:
ALTER TABLE table_name DROP CONSTRAINT cons_name;

Ex:

ALTER TABLE person DROP CONSTRAINT location_fk;

Use Of ON DELETE SET NULL and ON DELETE CASCADE:

When we use the column value of a parent table as foreign key in a child table, if the record in the parent table is deleted and the value still remains in the child table will lead to data inconsistency. Hence, database does not allow it . While creating a foreign key, we can specify what to do if we try to delete a value from a parent table which we have used as foreign key in a child table by using clauses like **ON DELETE SET NULL and ON DELETE CASCADE** .

ON DELETE SET NULL:

A foreign key with ON DELETE SET NULL means , if a record in the parent table is deleted then the corresponding records in the child table will be automatically set to NULL.

Syntax:

CREATE TABLE table_name (colname1 DATATYPE(size), colname2 DATATYPE(size), ...,
CONSTRAINT cons_name FOREIGN KEY (colname1,colname2) REFERENCES table1(col1,col2) ON DELETE SET NULL);

Ex:

CREATE TABLE production(suppl_id NUMBER(4),warehouse_id NUMBER(6), warehouse_name VARCHAR2(20),product_name VARCHAR2(20),qnty NUMBER(4),unit_price NUMBER(6,2),CONSTRAINT suppl_fk FOREIGN KEY(suppl_id) REFERENCES supplied(supp_id), CONSTRAINT warehouse_fk (warehouse_id,warehouse_name) REFERENCES warehouse (id,name) ON DELETE SET NULL);

The above example specifies that if at all the records available in the warehouse table are deleted, then automatically the **respective records in the production table (warehouse_id , warehouse_name)** (which are referring the values of the composite key warehouse(id,name) of the deleted record from warehouse) will be set to null.

Syntax:

ALTER TABLE table2 ADD CONSRAINT cons_name FOREIGN KEY (col4) REFERENCES table1(col1) ON DELETE SET NULL;

Ex: CREATE TABLE person(name VARCHAR2(30),dob DATE,gender CHAR(1), address NUMBER(4) ,gender CHAR(1));

ALTER TABLE person ADD CONSTRAINT location_fk FOREIGN KEY(address)
REFERENCES location(loc_id) ON DELETE SET NULL;

The above example specifies that if any record in the location table (loc_id) is / are deleted , then automatically the respective rec**ords from the person table** (which are REFERENCING the values of respective loc_id of the deleted record from Location table) will be set to NULL

ON DELETE CASCADE

A FOREIGN KEY with ON DELETE CASCADE means that if a record in the parent table is deleted , then the corresponding records in the child table will be deleted automatically .

Syntax :

CREATE TABLE tablename (col1 DATATYPE(size) CONSTRAINT cons_name REFERENCES table1(col1) ON DELETE CASCADE,col2 DATATYPE(size), col3 DATATYPE(size) , ...);

Ex: CREATE TABLE person(name VARCHAR2(30),dob DATE,gender CHAR(1),address NUMBER(4) REFERENCES location(loc_id) ON DELETE CASCADE, gender CHAR(1));

The above example specifies that, If the any record is deleted from the location table, then the corresponding **records from the person table** (which are depending on the values of loc_id of deleted record from location table) will be deleted automatically

Syntax:

CREATE TABLE table2 (col4 DATATYPE(size), col5 DATATYPE(size), col6 DATATYPE(size), ..., CONSTRAINT cons_name FOREIGN KEY (col4,col5) REFERENCES table1(col1,col2) ON DELETE CASCADE);

Ex: CREATE TABLE production(suppl_id NUMBER(4),warehouse_id NUMBER(6), warehouse_name VARCHAR2(20),product_name VARCHAR2(20),qnty NUMBER(4),unit_price NUMBER(6,2),CONSTRAINT suppl_fk FOREIGN KEY(suppl_id) REFERENCES supplied(supp_id),CONSTRAINT warehouse_fk (warehouse_id,warehouse_name) REFERENCES warehouse (id,name) ON DELETE CASCADE);

The above example specifies that, If any record is deleted from the warehouse table , then automatically the **records from production table**(which are referring the values of the composite key(id ,name) of the deleted record from the warehouse table) will be deleted automatically.

Syntax:

ALTER TABLE table2 ADD CONSRAINT cons_name FOREIGN KEY (col4) REFERENCES table1(col1) ON DELETE CASCADE;

Ex: CREATE TABLE person(name VARCHAR2(30),dob DATE,gender CHAR(1),
address NUMBER(4) ,gender CHAR(1));

ALTER TABLE person ADD CONSTRAINT location_fk FOREIGN KEY(address)
REFERENCES location(loc_id) ON DELETE CASCADE;

The above example specifies that , If the data is deleted from the location table location id , automatically they will be removed from person table address field.